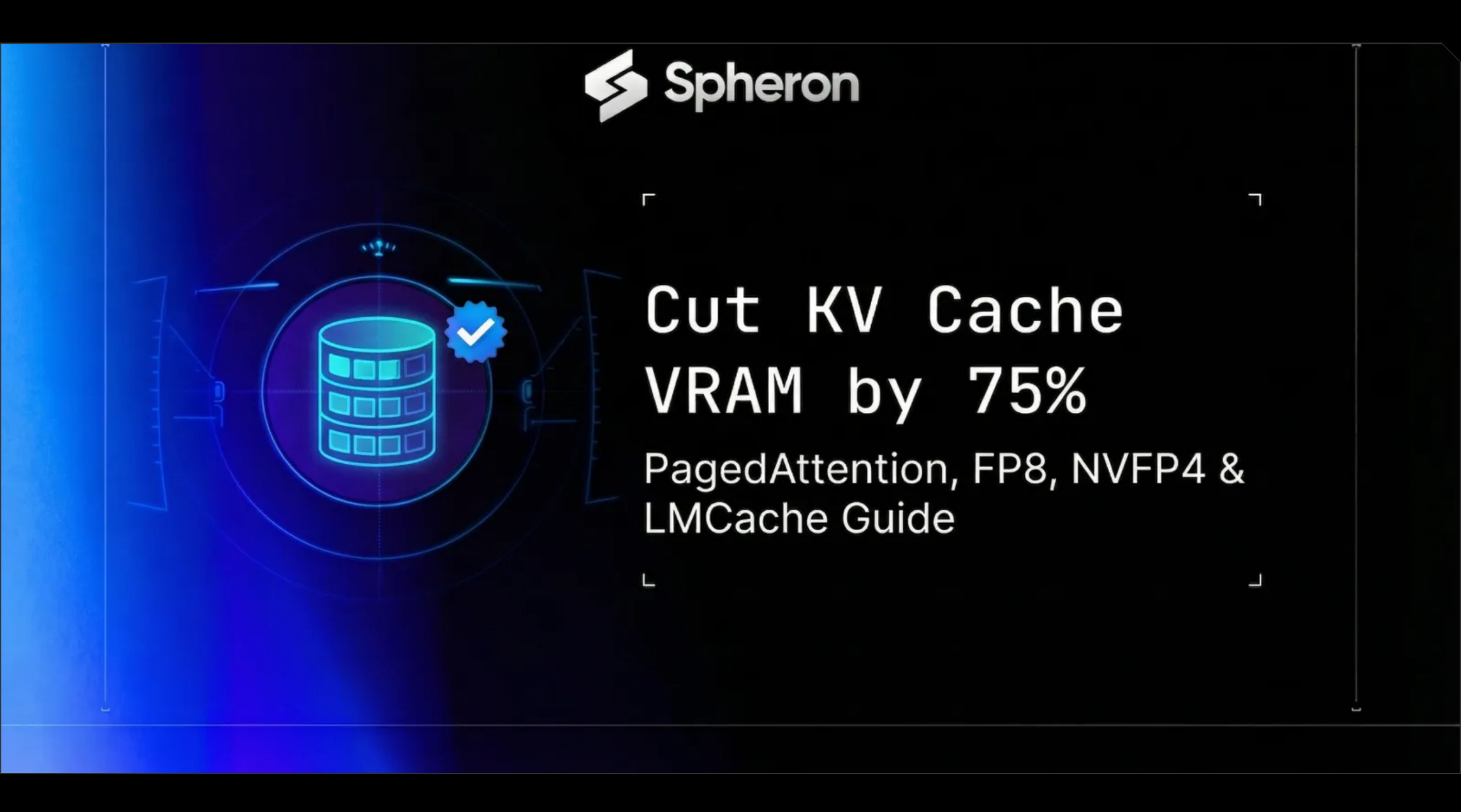
New cluster 64x B300 cluster is live. Reserve your block before someone else does. Bidding closes Oct 21. Reserve your block →

ENGINEERING

KV Cache Optimization: Serve 10x More Users per GPU (2026)

← BACK TO BLOG • WRITTEN BY MITRASISH, CO-FOUNDER & CTO, SPHERON • PUBLISHED MAR 28, 2026 | UPDATED OCT 3, 2026

KV Cache Optimization · LLM Inference · vLLM · PagedAttention · GPU Cloud · AI Infrastructure · GPU Memory · FP4 · FP8 · NVFP4


Spheron
Cut KV Cache VRAM by 75%
PagedAttention, FP8, NVFP4 & LMCache Guide

A single Llama 3 70B request at 128K context needs 42 GB of GPU memory just for the KV cache. That leaves almost nothing for model weights on an 80 GB card, and zero room for concurrent users. Before you provision more GPUs, there are five optimization techniques that compound on each other to cut that 42 GB to under 6 GB on the right hardware. For the full VRAM calculation context including model weights and activation memory, see GPU Memory Requirements for LLMs. If you're not sure whether your workload is memory-bound or compute-bound, start with understanding the memory wall in LLM inference before applying these optimizations. For the application-layer discipline of deciding what to put in context in the first place, see the context engineering guide for production agents.

TL;DR: How to Optimise the KV Cache

- **The problem:** one Llama 3 70B request at 128K context needs 42GB of VRAM for the KV cache alone, leaving almost nothing for weights on an 80GB card.
- **PagedAttention (vLLM):** removes fragmentation by paging the cache like virtual memory. The single biggest win, and it is on by default.
- **FP8/NVFP4 quantisation:** halves or quarters cache size at minimal quality cost.
- **CPU offloading and prefix caching:** LMCache reuses shared prefixes, which cuts time-to-first-token on long-context workloads.
- **Then size the GPU to the optimised budget**, not the naive one. Spheron H200 carries 141GB, from $3.32/hr as of 19 Oct 2026. Rent H200 →

Why the KV Cache Dominates Your VRAM Budget

During autoregressive generation, the model stores key and value vectors for every previously generated token so it doesn't recompute them on each new token. These tensors accumulate in the KV cache throughout the lifetime of a request and are only freed when the request completes.

The KV cache grows in five dimensions simultaneously: number of transformer layers, number of KV heads, head dimension, sequence length, and concurrent batch size. At short contexts (under 4K tokens), it's negligible. At long contexts (32K-128K tokens) with real concurrent load, it becomes the dominant memory consumer.

Here's how KV cache memory scales for Llama 3.1 70B at BF16:

Context Length	1 Concurrent User	8 Concurrent Users
4,096 tokens	~1.3 GB	~10.7 GB
16,384 tokens	~5.4 GB	~43 GB
32,768 tokens	~10.7 GB	~86 GB
131,072 tokens	~42.9 GB	~343 GB

The model weights for Llama 3.1 70B at FP16 are ~140 GB. At 32K context with 8 concurrent users, the KV cache alone exceeds the weights. At 128K context with 8 users, the KV cache is 2.4x the model size.

The KV Cache Memory Formula

The exact formula for KV cache memory:

```
KV_bytes = 2 × L × H_kv × D × S × B × bytes_per_element
```

Where:

- `L` = number of transformer layers
- `H_kv` = number of key-value heads (after GQA)
- `D` = head dimension
- `S` = sequence length (tokens)
- `B` = concurrent batch size
- `bytes_per_element` = 2 for BF16/FP16, 1 for FP8, 0.5 for FP4

Worked examples for three common models:

Model	Layers	KV Heads	Head Dim	KV/token at BF16
Llama 3.1 8B	32	8	128	~0.131 MB
Llama 3.1 70B	80	8	128	~0.327 MB
Llama 3.1 405B	126	8	128	~0.516 MB

Llama 3 uses Grouped Query Attention (GQA) with 8 KV heads instead of 64 query heads. That 8x reduction in KV heads cuts KV cache size by 8x vs standard multi-head attention (MHA). A comparable MHA model would need ~2.6 MB per token at BF16 for 70B scale. That 8x is specific to Llama 3, though, not to GQA as a category: Gemma 2 is also a GQA model and only cuts its cache 2x. Our GQA vs MHA KV cache memory savings breakdown reads `num_key_value_heads` out of seven released models to show where each one actually lands.

One caveat: the formula applies to full-context attention. Models with sliding window attention (Mistral's Longformer layers, for example) use a smaller effective window per layer, so the actual per-layer KV cache is bounded by the window size, not the full context length. Llama 4 Scout takes a similar approach with its iRoPE design, interleaving chunked local-attention layers with periodic global ones; see our Llama 4 VRAM and KV cache breakdown for how that architecture is what makes its 10M-token context window remotely serviceable.

For the full weight memory formula including quantization formats, see GPU Memory Requirements for LLMs.

PagedAttention in vLLM: Virtual Memory for the KV Cache

The naive approach to KV cache allocation reserves the maximum context-length memory for each request at the moment it arrives. A request with a 32K max context gets 32K worth of KV blocks reserved upfront, even if it only generates 500 tokens. In practice, 60-80% of that reserved memory goes unused.

PagedAttention solves this by treating KV cache like OS virtual memory. The cache is split into fixed-size blocks (16 tokens per block by default). Blocks are allocated on-demand as tokens are generated and freed immediately when requests complete. No wasted pre-allocation, no fragmentation from variable-length requests.

The throughput gain is not marginal. Going from naive allocation to PagedAttention typically doubles or quadruples the number of concurrent requests you can serve on the same GPU, because you stop reserving memory that never gets used.

PagedAttention is enabled by default in vLLM. The launch command:

```bash
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-70B-Instruct \
  --gpu-memory-utilization 0.90 \
  --max-model-len 32768 \
  --kv-cache-dtype auto
```

`--gpu-memory-utilization 0.90` tells vLLM to use 90% of available VRAM (after weights load) for the KV cache pool. `--max-model-len` caps the context window, which controls the maximum KV cache block pool size vLLM allocates. Set it to your actual maximum context need, not the model's theoretical maximum.

For a complete multi-GPU vLLM deployment setup, see vLLM Multi-GPU Production Deployment 2026. For how PagedAttention interacts with continuous batching and chunked prefill to maximize overall scheduling throughput, see the LLM serving optimization guide.

KV Cache Quantization: FP8 and NVFP4

KV tensors can be stored at lower precision than the model weights. The attention computation is relatively tolerant of precision reduction because the softmax normalization in attention averages out small rounding errors.

FP8 on H100/A100: Switch from BF16 to FP8 KV storage with one flag:

```bash
--kv-cache-dtype fp8
```

This halves KV cache memory vs BF16. Quality loss is negligible for most production workloads. FP8 KV cache works on both H100 and A100. Note: on A100, this is a software-level memory optimization. vLLM stores KV tensors in FP8 and dequantizes to BF16 before the attention computation. You get the memory reduction, but the A100 has no hardware FP8 Tensor Cores, so there is no throughput gain from the quantization itself.

NVFP4 on Blackwell (B200, RTX 5090): For an additional 50% reduction vs FP8:

```bash
--kv-cache-dtype nvfp4
```

This requires Blackwell hardware. On H100 or A100, this flag will either error or behave incorrectly depending on vLLM version. Do not use it on Hopper or Ampere GPUs.

KV cache memory for Llama 3.1 70B at 32K context, 8 concurrent users:

Precision	KV Cache Size	Savings vs BF16
BF16	~85.9 GB	baseline
FP8	~42.9 GB	50%
NVFP4	~21.5 GB	75%

The quality tradeoff: FP8 KV is safe for production across virtually all tasks. NVFP4 KV should be validated on your specific task before deploying to production, particularly for reasoning-heavy workloads. For the full quality analysis of NVFP4, see FP4 Quantization on Blackwell GPUs.

Reasoning models amplify KV cache pressure significantly. For distilled reasoning models based on standard attention (like DeepSeek-R1-Distill-Llama-70B), a single 30,000-token reasoning chain uses roughly 9 GB of KV cache at FP16. FP8 KV quantization cuts that to ~4.5 GB, which is one of the highest-impact optimizations for chain-of-thought workloads. Note: the full DeepSeek-R1-671B model uses Multi-head Latent Attention (MLA), which compresses KV cache to a latent dimension and results in a much smaller footprint (~2 GB for the same context length). See our guide to reducing reasoning model inference costs for techniques specific to reasoning workloads, and Inference-Time Compute Scaling on GPU Cloud for how extended reasoning changes GPU instance sizing and autoscaling strategy.

Note: hardware FP4 tensor core acceleration requires Blackwell. On H100, vLLM can load NVFP4-quantized *model weights* via Marlin software fallback, but `--kv-cache-dtype nvfp4` for KV storage acceleration is a Blackwell-only feature.

CPU KV Cache Offloading

When GPU KV cache fills during a traffic spike, vLLM's default behavior is to queue or reject new requests. Adding a CPU swap space prevents this by using system RAM as a secondary KV cache tier.

```bash
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-70B-Instruct \
  --kv-cache-dtype fp8 \
  --swap-space 32
```

`--swap-space 32` reserves 32 GB of system RAM. When GPU KV cache pressure is high, low-priority request blocks swap to CPU RAM. When those requests need their blocks again, they swap back.

The cost: each eviction/reload adds 2-10ms of latency depending on PCIe bandwidth. For a latency-sensitive application serving strict SLAs, this is not acceptable for the hot path. For a batch processing workload or an application where occasional latency spikes are tolerable, it's the right call over request rejection.

Starting configuration: `--swap-space 16` for moderate workloads, `--swap-space 32` for bursty traffic. You need a host with at least 64 GB of DRAM to leave enough room for the OS and other processes after the swap reservation.

Full CPU offloading (loading model layers into CPU RAM) is different from swap space and not recommended for production serving given the 50x bandwidth penalty between CPU RAM and GPU HBM.

For workloads that need more capacity than CPU DRAM can provide, see NVMe KV Cache Offloading for LLM Inference for deploying LMCache with a disk backend. For the hardware-accelerated version of this same pattern using BlueField-4 DPUs and cuFile, see the ICMSP architecture and setup guide.

LMCache and Prefix Caching: Cutting TTFT for Long-Context Workloads

For production multi-node setups where KV caches move between GPU nodes, see NVIDIA NIXL and Disaggregated Inference for wire-speed transfer details.

Many production LLM workloads share a common prefix across requests: a long system prompt, a shared reference document in a RAG pipeline, or a conversation history. Without prefix caching, every request recomputes the KV cache for that prefix from scratch during prefill, which is the expensive step for long contexts.

vLLM's built-in `--enable-prefix-caching` flag detects when a new request's prefix matches a previously-computed prefix and serves it from the in-memory cache. This is zero-configuration and applies within a single server session.

LMCache extends this to persist cached KV blocks across server restarts and multiple server instances using Redis or disk backends. The practical gain is dramatic for long-context workloads: on a 128K-token system prompt on H100, TTFT drops from ~11s to ~1.5s because the full prefill computation is replaced by a cache hit. For a step-by-step walkthrough of deploying LMCache for distributed KV sharing across multiple vLLM workers on Kubernetes, see the dedicated deployment guide. For multi-node disaggregated serving with a dedicated KV memory pool, Mooncake's KVCache-centric architecture takes a different approach than LMCache: it routes KV blocks from prefill workers to decode workers through a distributed store rather than broadcasting from one origin to many readers.

For Kubernetes-native KV cache-aware routing across GPU pools, see the llm-d deployment guide which covers the Gateway API Inference Extension's cache-aware scheduler.

At the cluster routing layer, GKE Inference Gateway's KV-cache-aware load balancer uses per-replica prefix hit metrics to route requests to the replica most likely to serve them from cache.

Install and launch:

```bash
pip install lmcache
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-70B-Instruct \
  --enable-prefix-caching \
  --kv-cache-dtype fp8
```

When prefix caching helps most:

- RAG pipelines where the same document or knowledge base is prepended to many queries
- Chatbots with long, fixed system prompts (multi-paragraph instructions)
- Batch processing where many items share an identical preamble
- Multi-turn conversations where the growing history is re-submitted each turn

LMCache compatibility depends on the vLLM version you're running. Check the LMCache documentation for version-specific setup instructions before deploying.

KV cache transfer between nodes is also the key mechanism behind prefill-decode disaggregation, where cache is produced on a prefill node and consumed on a separate decode node. For workloads with long-lived shared context, pre-populating KV caches during idle time (sleep-time compute) extends this further by eliminating prefill cost at query time entirely.

GQA and Flash Attention: Architecture-Level Optimizations

Both of these are baked into modern models and serving frameworks. You don't configure them, but understanding what they do explains why the KV cache numbers above are already much smaller than they could be.

Grouped Query Attention (GQA): Llama 3 uses 8 KV heads for every 64 query heads. Each query head attends to the same 8 KV heads during attention computation, so you only need to store 8 KV heads worth of tensors per layer instead of 64. This is an 8x KV cache reduction vs standard Multi-Head Attention (MHA) for Llama-scale models.

Multi-Query Attention (MQA) takes this further: 1 KV head shared by all query heads. Used in some smaller models for maximum memory efficiency at some quality cost.

KV head counts for popular 2026 models:

Model	Query Heads	KV Heads	KV Cache Reduction vs MHA
Llama 3.1 8B	32	8	4x
Llama 3.1 70B	64	8	8x
Llama 3.1 405B	128	8	16x
Mistral 7B	32	8	4x
Qwen 2.5 72B	64	8	8x

Flash Attention: Rewrites the attention kernel to avoid materializing the full NxN attention matrix in GPU HBM. Instead of writing the full matrix to memory and reading it back, Flash Attention computes attention in tiles that stay in SRAM. For long sequences, this reduces HBM reads/writes by 5-20x. Flash Attention is the default in vLLM and TensorRT-LLM and requires no configuration. On Blackwell GPUs (B200, B300), FlashAttention-4 replaces FA3 with an SM100 tile-based kernel that cuts KV cache read overhead further at long context lengths. See the FlashAttention-4 guide for how FA4's SM100 tiling directly reduces the KV cache read overhead at long context lengths. If you're still on Hopper, the FlashAttention 2 vs FlashAttention 3 comparison breaks down the throughput gap on H100 and H200 before you plan a Blackwell migration.

For end-to-end throughput comparisons across inference engines that include all of these optimizations, see vLLM vs TensorRT-LLM vs SGLang Benchmarks.

GPU Selection Guide: Match VRAM to Your Optimized KV Cache Budget

The right GPU is the one where: model weights + optimized KV cache + 10% activation headroom fits in VRAM. Optimizing the KV cache first, then sizing the GPU, is cheaper than throwing more VRAM at an unoptimized stack. For a broader cost reduction framework that goes beyond KV cache, see the GPU cost optimization playbook. For application-layer cost reduction on top of KV cache, see Semantic Caching for LLM Inference. For teams where power availability is capping expansion before budget does, see AI data center power constraints in 2026 for power-aware capacity planning.

Three workload profiles showing the calculation:

Light workload: Llama 3.1 8B, 8K context, 16 concurrent users, FP8 KV cache

- Weights (FP8): ~8 GB
- KV cache (FP8): 2 × 32 × 8 × 128 × 8192 × 16 × 1 = ~8.6 GB
- Total with headroom: ~18 GB
- Fits on: A100 80GB with 62 GB to spare for larger batches

Medium workload: Llama 3.1 70B, 32K context, 8 users, FP8 KV cache

- Weights (FP8): ~70 GB
- KV cache (FP8): 2 × 80 × 8 × 128 × 32768 × 8 × 1 = ~42.9 GB
- Total: ~113 GB, needs 2x H100 at FP8 weights + FP8 KV
- With NVFP4 KV on Blackwell: ~70 GB + ~21.5 GB = ~91.5 GB, fits B200 (192 GB) with room

Heavy workload: Llama 3.1 70B, 128K context, 8 users, FP8 KV cache

- Weights (FP8): ~70 GB
- KV cache (FP8): 2 × 80 × 8 × 128 × 131072 × 8 × 1 = ~171 GB
- Total: ~241 GB, requires 4x H100 or 2x H100 with further prefix caching to reduce active context

GPU selection with live Spheron pricing, resolved on every render:

Workload Profile	VRAM Needed	Recommended GPU	Spheron Price
8B model, 8K context, 16 users, FP8 KV	~18 GB	A100 80GB SXM4	$1.48/hr
70B model, 32K context, 8 users, FP8 KV	~113 GB	2x H100 SXM5	$5.96/hr
70B model, 128K context, 4 users, FP8 KV	~156 GB	3x H100 SXM5	$8.94/hr
70B model, 32K context, 8 users, NVFP4 KV	~92 GB	B200 SXM6	$5.15/hr (spot)

> Pricing fluctuates based on GPU availability. Spheron rates above are live as of **19 Oct 2026**. Check current GPU pricing → for live rates.

L40S is not currently available in the Spheron GPU catalog. For L40S comparisons, see Best GPU for AI Inference in 2026 which covers a broader GPU-to-workload matching guide.

Benchmark Results: Throughput Gains from KV Cache Optimization

Stacked optimization gains for Llama 3.1 70B FP8 on H100 PCIe, starting from a naive baseline (rows 1-4 on H100 PCIe; final row on Blackwell):

Configuration	Concurrent Users	Throughput (tok/s)	KV Cache VRAM	Notes
Naive pre-alloc, BF16 KV	4	~400	~40 GB	No PagedAttention
+ PagedAttention (vLLM default)	8	~900	~22 GB	2x users, same GPU
+ FP8 KV cache	16	~1,800	~11 GB	4x users vs baseline
+ Prefix caching (LMCache)	16	~1,800 (TTFT: 1.5s vs 11s)	~11 GB	Latency win, not throughput
+ NVFP4 KV (Blackwell only)	32	~2,600	~5.5 GB	8x users vs baseline

These are estimated figures based on published vLLM and LMCache benchmark data, not independently reproduced results on this hardware configuration. They are intended to illustrate relative ordering and magnitude of gains, not precise throughput you should expect. Actual results depend on model, hardware, batch composition, and request length distribution. Run your own benchmarks on your target hardware before making infrastructure decisions.

Quick Reference: KV Cache Optimization Checklist

1. Enable PagedAttention (default in vLLM, on by default)
2. Set `--gpu-memory-utilization 0.90` (raise to 0.92-0.95 if model fits tightly)
3. Use FP8 KV cache on H100/A100: `--kv-cache-dtype fp8`
4. Use NVFP4 KV cache on Blackwell only: `--kv-cache-dtype nvfp4`
5. Enable prefix caching: `--enable-prefix-caching`
6. Set `--swap-space 16` for burst tolerance on bursty workloads
7. Choose GQA-native models (Llama 3 family) for long-context serving
8. Verify Flash Attention is active (on by default in vLLM)
9. Size your GPU to the optimized budget, not the theoretical maximum

The DeepSeek V4 deployment guide covers KV cache tuning for large MoE models like the 1.6T-parameter V4-Pro and 284B V4-Flash specifically, including the interaction between expert parallelism and KV cache allocation.

If you're hitting KV cache limits at long contexts, state space models like Mamba-3 eliminate the KV cache entirely at the cost of some accuracy tradeoffs. Liquid AI's LFM family takes a similar constant-memory approach: LFM2 inference on GPU cloud covers the VRAM math and throughput profile for their structured-operator architecture.

For context lengths beyond 1M tokens where KV offloading itself becomes the bottleneck, see Ring Attention and Striped Attention for 10M-token inference for how to distribute the attention computation itself across a multi-GPU ring.

> KV cache optimization is what makes production LLM inference financially viable. Once you have your optimized VRAM number, pick the right-sized Spheron GPU and avoid paying for capacity you don't need.
>
> H100 SXM5 on Spheron → | Check A100 availability → | View all GPU pricing →

STEPS / 06

Quick Setup Guide

01 **Calculate your KV cache memory requirements**

Use the formula: KV_cache_bytes = 2 × num_layers × num_kv_heads × head_dim × seq_len × batch_size × bytes_per_element. For Llama 3.1 70B at BF16 with 128K context and one concurrent user: 2 × 80 × 8 × 128 × 131072 × 1 × 2 = ~42.9 GB. Multiply by your target concurrent users for total KV cache budget.

02 **Enable PagedAttention in vLLM**

PagedAttention is enabled by default in vLLM. The key parameter is --gpu-memory-utilization (default 0.90), which controls how much of remaining VRAM (after model weights load) vLLM reserves for the KV cache pool. Set --max-model-len to your target context length to prevent vLLM from reserving memory for longer contexts than you need.

03 **Apply KV cache quantization (FP8 or NVFP4)**

On H100 and A100, add --kv-cache-dtype fp8 to halve KV cache VRAM vs BF16 with minimal quality impact. On Blackwell GPUs (B200, RTX 5090), use --kv-cache-dtype nvfp4 for a further 50% reduction vs FP8. Do not use --kv-cache-dtype nvfp4 on H100 or A100 as it requires Blackwell hardware for acceleration.

04 **Configure CPU KV cache offloading for burst tolerance**

Set --swap-space 32 in your vLLM launch command to reserve 32 GB of system RAM as a secondary KV cache tier. When GPU KV cache fills during traffic spikes, low-priority request blocks swap to CPU RAM instead of triggering OOM errors. This adds 2-10ms swap latency per eviction but prevents request rejection. Requires a host with 64+ GB DRAM.

05 **Enable prefix caching with LMCache**

Install LMCache (pip install lmcache) and add --enable-prefix-caching to your vLLM launch command. For RAG pipelines or chatbots with long shared system prompts, this reduces TTFT from seconds to under 2s by serving the shared prefix from cache. LMCache compatibility depends on your vLLM version; check the LMCache documentation for version-specific setup.

06 **Match GPU tier to your optimized VRAM budget**

After applying optimizations, recalculate: total VRAM = model weights + optimized KV cache + 10% headroom. Use the smallest Spheron GPU tier that fits this number. For Llama 3.1 70B at 32K context with 8 users and FP8 KV: ~70 GB weights + ~42.9 GB KV = ~113 GB, requiring 2x H100 SXM5. Multiply the live per-GPU H100 rate on the pricing page by two for the hourly cost.

FAQ / 05

Frequently Asked Questions

01 **How much GPU memory does KV cache use for Llama 3 70B at 128K context?** ×

A single Llama 3.1 70B request at 128K context uses approximately 42.9 GB for the KV cache at BF16 precision. The formula is: 2 × 80 layers × 8 KV heads × 128 head_dim × 131072 tokens × 2 bytes = ~42.9 GB. With FP8 KV quantization this drops to ~21.5 GB, and with NVFP4 on Blackwell hardware to ~10.7 GB. On an 80 GB GPU, serving multiple concurrent users at this context length requires FP8 or NVFP4 KV quantization plus multi-GPU setup.

02 What is PagedAttention and how does it reduce KV cache memory waste? +

03 What is NVFP4 KV cache quantization and which GPUs support it? +

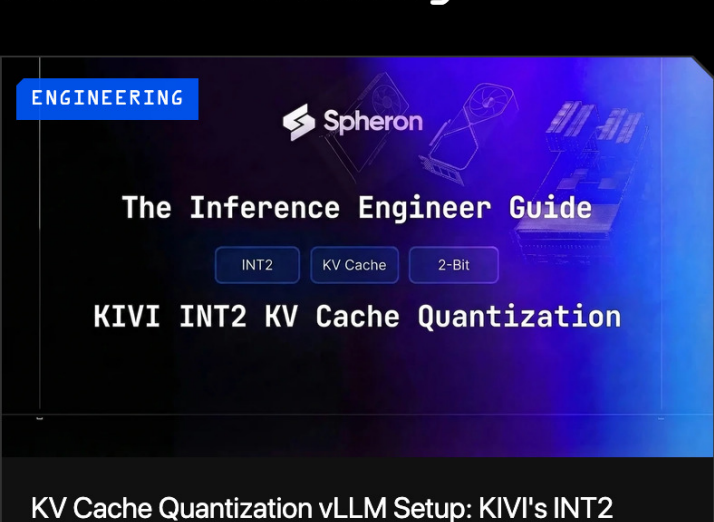
04 How does LMCache reduce TTFT from 11s to 1.5s? +

05 Can I serve production LLM traffic on an A100 80GB after KV cache optimization? +

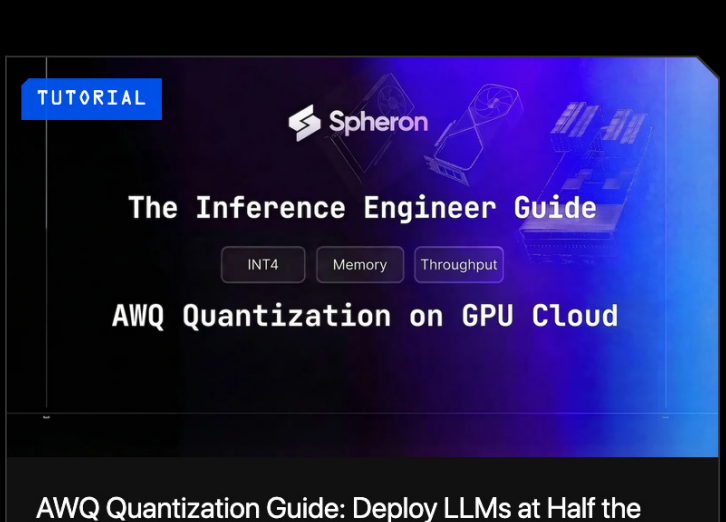
← Back to all posts

Related Reading

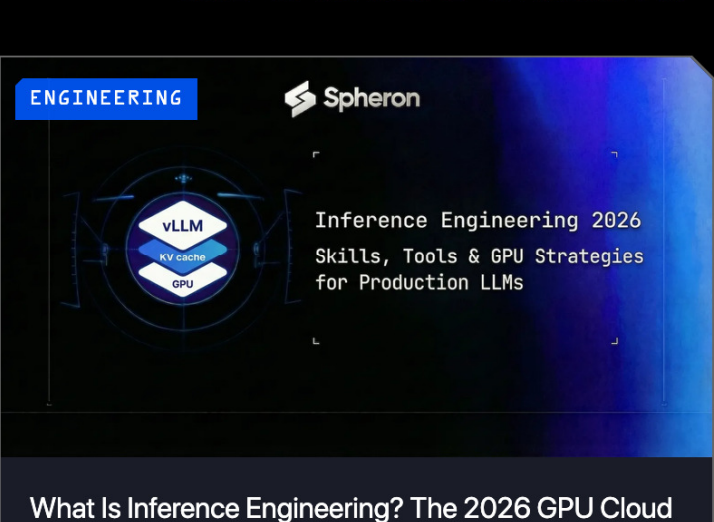
MORE ON INFERENCE OPTIMIZATION →



KV Cache Quantization vLLM Setup: KIVI's INT2 Method (2026)
Sep 5, 2026



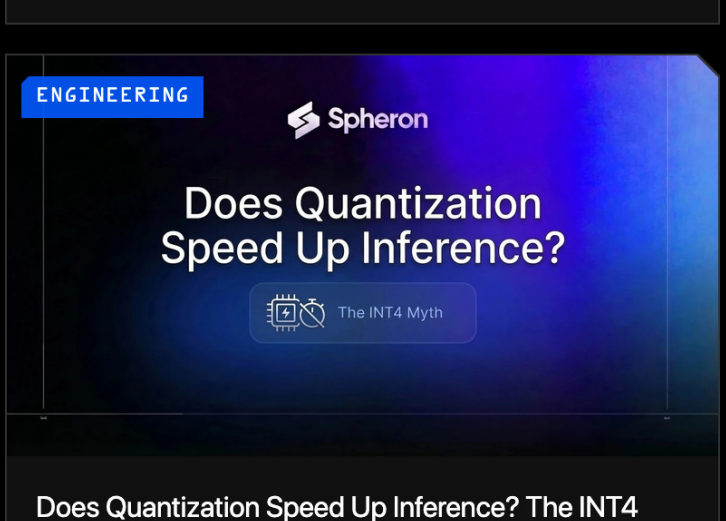
AWQ Quantization Guide: Deploy LLMs at Half the GPU Cost (2026)
Apr 7, 2026


ENGINEERING
Spheron
vLLM
KV Cache
GPU
Inference Engineering 2026
Skills, Tools & GPU Strategies for Production LLMs

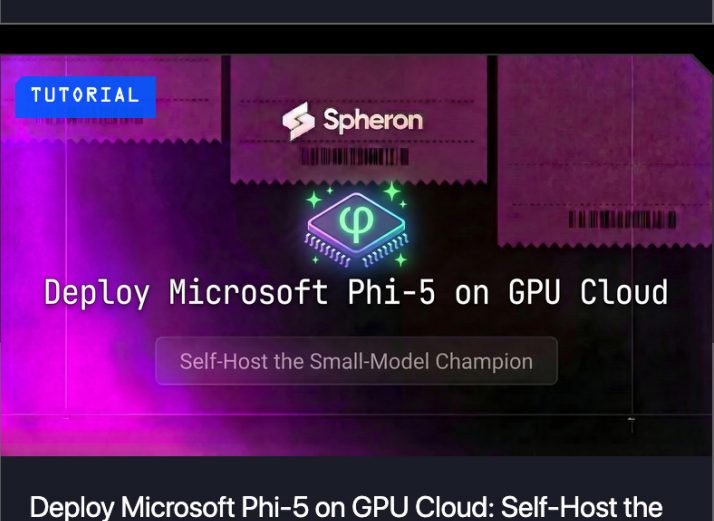
What Is Inference Engineering? The 2026 GPU Cloud Guide
Apr 7, 2026

ENGINEERING
Spheron
LLM Inference Too Slow?
7 causes & fixes: VRAM, batching, FP8, FlashAttention

Why Your LLM Inference Is Slow (And How to Fix It)
Mar 29, 2026



Does Quantization Speed Up Inference? The INT4 Myth
Sep 16, 2026


TUTORIAL
Spheron
Deploy Microsoft Phi-5 on GPU Cloud
Self-Host the Small-Model Champion

Deploy Microsoft Phi-5 on GPU Cloud: Self-Host the Small-Model Champion for Cost-Efficient Inference...
May 26, 2026

• TRY IT YOURSELF

TRY IT ON REAL GPUS

The GPUs behind these guides are the ones you can rent here: H100s, H200s, B200s, and more, billed per minute after a 20-minute minimum runtime, with no contracts. Pick one and you are live in under two minutes.

Rent a GPU

DEPLOY TIME < 2 min
UPTIME SLA 99.9%
GPU MODELS 10+
STARTING AT $0.48/hr

GLOBAL COMPUTE, BROUGHT TO YOU BY
Spheron
Bare-metal GPU infrastructure for AI teams that need scale, speed, and simplicity.
Deploy Now
Contact Sale →

PRODUCT
App
Pricing
Customers
GPU Catalog
Supply GPU

RESOURCES
Documentation
API Reference
Blog
GitHub
Changelog

TOOLS
GPU Recommender
Training Cost Calculator

FEATURES
On-Demand Instances
Spot Instances
Reserved Commitments

COMMUNITY
Discord
X (Twitter)
LinkedIn
YouTube
Telegram

GPUS
NVIDIA Vera Rubin
NVIDIA GB300
NVIDIA GB200
NVIDIA B300
NVIDIA H100
NVIDIA B200
NVIDIA H200
NVIDIA A100
NVIDIA GH200
NVIDIA L40S
AMD MI300X